

Payment Architecture & Razorpay Deep Dive

Complete technical specification of checkout workflows, synchronous client validation, resilient webhook event processing, idempotency guarantees, and MongoDB ACID state consistency.

Platform: CraftyWrap E-Commerce **Payment Gateway:** Razorpay API v1 & Webhooks

Stack: Next.js • Express • Mongoose (Replica Sets) **Author:** Lead Full-Stack Engineering Team

Classification: Internal Architecture & Interview Spec

01 Executive Architecture Summary

Payment backend design is the most critical and vulnerability-prone tier of an e-commerce platform. In **CraftyWrap**, checkout is architected as a **hybrid dual-path verification model**:

1. Fast-Path Client Verification (UX)

- Triggered directly from the browser Razorpay SDK callback.
- Validates `razorpay_payment_id` and `razorpay_signature` against `RAZORPAY_KEY_SECRET`.
- Transitions the order status immediately, providing instant visual feedback and preventing user bounce.

2. Asynchronous Webhook Engine (Truth)

- Direct, server-to-server HTTP POST emitted by Razorpay upon payment capture.
- Operates independently of whether the user closes the tab, loses network, or suffers a browser crash.
- Guarantees inventory deduction, promo usage, and financial fulfillment even in worst-case client dropouts.

Core Engineering Principle: The Dual-Verification Invariant

Both the client verification endpoint and the webhook pipeline converge onto an **idempotent, atomic transaction**. Regardless of whether the client or the webhook reaches the server first (or if both arrive simultaneously), stock is deducted *exactly once*, promo codes are recorded *exactly once*, and the order is marked *paid* safely.

02 End-to-End Payment Flow Topology

1 Customer Submits Checkout Form (Client)

The user inputs their shipping address and clicks "Pay". The Next.js frontend posts shipping information and cart ownership tokens to `POST /api/orders/checkout`.

2 Order Staging & Razorpay Order Inception (Backend)

The backend sanitizes the cart, verifies live inventory stock, computes charges (Subtotal + ₹50 Shipping + ₹49 Gift Wrap – Promo Discount), generates a sequential order number (`CW-2026-XXXXXX`), creates an order with `orderStatus: 'payment_pending'` , and invokes `razorpay.orders.create()` .

3 Razorpay Checkout Modal Invocation (Client SDK)

Frontend dynamically mounts `https://checkout.razorpay.com/v1/checkout.js` . The SDK presents UPI (GPay, PhonePe), Cards, and Netbanking rails using CraftyWrap brand parameters (`#5C3A21`).

4A Synchronous Path: Client Signature Submission

On payment authorization, Razorpay passes `razorpay_payment_id` and `razorpay_signature` to the SDK handler, which posts to `POST /api/orders/:orderId/verify-payment` .

4B Asynchronous Path: Server-to-Server Webhook Delivery

Razorpay fires `payment.captured` and `order.paid` directly to `POST /api/razorpay/webhook` . This occurs asynchronously in the background.

5 Atomic Fulfillment & Cart Invalidation

MongoDB replica session executes: updates `paymentStatus = 'paid'` , transitions `orderStatus = 'preparing'` , decrements stock via `$inc: { stockCount: -quantity }` , and purges the active cart.

03 Order Staging & Razorpay Orders API

When `createOrderFromCart` in `backend/src/services/orderService.ts` is invoked, it adheres to defensive financial rules before involving the payment gateway:

Cart Sanitization: Any ghost items with zero quantity or inactive/deleted product IDs are purged automatically before price calculations.

Live Stock Verification: Ensures every cart item's quantity does not exceed current `stockCount`. If insufficient, throws HTTP 409 Conflict.

Financial Exactness: Currency conversion enforces the Razorpay paise rule (1 INR = 100 Paise). Amounts are converted using `Math.round(total * 100)` to eliminate floating point truncation issues.

Atomic Sequential Numbering: Generates human-readable identifiers such as `CW-2026-000001` via a dedicated atomic MongoDB counter collection.

Razorpay API Request Specification

```
const orderPayload = {
  amount: Math.round(total * 100), // e.g., ₹1,499.00 -> 149900 paise
  currency: 'INR',
  receipt: orderNumber,           // e.g., "CW-2026-000042"
  notes: { craftywrapOrderNumber: orderNumber },
};
const razorpayOrder = await razorpay.orders.create(orderPayload);
createdOrder.paymentDetails = { razorpayOrderId: razorpayOrder.id };
await createdOrder.save();
```

Graceful Test/Demo Fallback

If the backend detects that `RAZORPAY_KEY_ID` or `RAZORPAY_KEY_SECRET` is unconfigured (such as in local preview environments or isolated CI runs), it assigns a virtual demo order identifier (`order_demo_...`). The frontend detects this and opens an interactive modal simulator rather than failing, ensuring smooth development and testing.

04 Client-Side Razorpay Integration (Checkout.js)

Rather than redirecting the user to an external checkout domain, CraftyWrap loads the script asynchronously via `loadRazorpayScript()` and opens the embedded Razorpay checkout modal.

```
const razorpayOptions = {
  key: rzpOrder.keyId,
  amount: rzpOrder.amount,
  currency: rzpOrder.currency,
  name: 'CraftyWrap',
  description: `Order #${formattedOrder.orderNumber}`,
  image: '/logo.png',
  order_id: rzpOrder.id,
  prefill: { name: fullName, phone: phone },
  theme: { color: '#5C3A21' }, // CraftyWrap signature warm brown
  handler: async function (res) {
    await apiRequest(`/orders/${order.id}/verify-payment`, {
      method: 'POST',
      body: JSON.stringify({
        razorpayPaymentId: res.razorpay_payment_id,
        razorpaySignature: res.razorpay_signature,
      }),
    });
  }
};
```

05 Synchronous Verification Logic

When the client posts to `POST /api/orders/:orderId/verify-payment`, `verifyRazorpayPayment` executes cryptographic verification:

```
// 1. Generate expected HMAC-SHA256 signature
const expectedSignature = crypto
  .createHmac('sha256', secret)
  .update(`${razorpayOrderId}|${razorpayPaymentId}`)
  .digest('hex');

// 2. Constant-time equality check to prevent timing attacks
const expectedBuf = Buffer.from(expectedSignature);
const actualBuf = Buffer.from(razorpaySignature);

if (expectedBuf.length !== actualBuf.length || !crypto.timingSafeEqual(expectedBuf, actualBuf)) {
  throw new HttpError(400, 'Razorpay payment signature is invalid.');
```

06 Deep Dive: The Razorpay Webhook Engine

The webhook engine located in `backend/src/controllers/razorpayWebhookController.ts` and `backend/src/services/razorpayWebhookService.ts` is the most technically nuanced component of the payment pipeline.

1. Raw Body Capture via Express Middleware

HMAC verification fails if the payload has been parsed and re-serialized because key order, whitespace, or escape sequences might change. In `backend/src/app.ts`, CraftyWrap preserves the exact incoming binary buffer:

```
app.use(
  express.json({
    limit: '50mb',
    verify: (req, _res, buf) => {
      if (buf && buf.length) {
        (req as any).rawBody = buf;
      }
    },
  }),
);
```

2. Webhook Signature Verification

Razorpay provides the `X-Razorpay-Signature` header. CraftyWrap validates it with:

```
export function verifyWebhookSignature(rawBody: Buffer | string, signature: string): boolean {
  const secret = process.env.RAZORPAY_WEBHOOK_SECRET;
  const expectedSignature = crypto.createHmac('sha256', secret).update(rawBody).digest('hex');
  return crypto.timingSafeEqual(Buffer.from(expectedSignature), Buffer.from(signature));
}
```

3. Strict Webhook Idempotency

Razorpay enforces an at-least-once delivery guarantee. If CraftyWrap's response takes longer than 5 seconds or encounters transient gateway network blips, Razorpay retries the webhook up to multiple times. Without an idempotency layer, stock could be decremented twice and promo counts corrupted.

CraftyWrap implements a persistent log backed by the `WebhookEvent` model (`backend/src/models/WebhookEvent.ts`):

```
// Query indexed unique eventId before triggering any business logic
const alreadyProcessed = await isEventProcessed(eventId);
if (alreadyProcessed) {
  return { status: 'already_processed', message: 'Event already processed.' };
}
```

4. Amount Mismatch & Underpayment Guard

A critical attack vector in e-commerce is the "underpayment exploit," where a modified client attempts to satisfy a ₹10,000 order by paying ₹1.00 on a tampered Razorpay order. CraftyWrap's webhook engine

performs an explicit mathematical sanity check:

```
const expectedPaise = Math.round(order.total * 100);
if (amountInPaise !== undefined && amountInPaise !== expectedPaise) {
  order.paymentDetails.failureReason = `Webhook amount mismatch: expected ${expectedPaise}, received ${amountInPaise}`;
  await order.save();
  return { status: 'failed', message: 'Payment amount mismatch.' };
}
```

07 State Consistency & MongoDB ACID Transactions

When payments are confirmed (either via synchronous verification or webhook), inventory, promotions, orders, and carts must transition simultaneously or not at all. CraftyWrap executes this inside a **Mongoose Session Transaction**:

```
const session = await mongoose.startSession();
try {
  await session.withTransaction(async () => {
    const currentOrder = await Order.findById(order._id).session(session);
    const isFirstPaymentVerification = currentOrder.paymentStatus !== 'paid';

    if (isFirstPaymentVerification) {
      // Atomic stock decrement with condition guard
      for (const item of currentOrder.items) {
        await Product.findOneAndUpdate(
          { _id: item.product, stockCount: { $gte: item.quantity } },
          { $inc: { stockCount: -item.quantity } },
          { new: true, session }
        );
      }
      // Increment promo usage count
      if (currentOrder.promoCodeApplied) {
        await PromoCode.updateOne(
          { code: currentOrder.promoCodeApplied },
          { $inc: { usedCount: 1 } },
          { session }
        );
      }
    }

    // Update order state
    currentOrder.paymentStatus = 'paid';
    currentOrder.orderStatus = 'preparing';
    currentOrder.paymentDetails.paidAt = new Date();
    await currentOrder.save({ session });

    // Invalidate active cart
    await Cart.updateOne(cartFilter, { $set: { items: [] } }, { session });
  });
} finally {
  await session.endSession();
}
```

08 Lifecycle State Machine Matrix

PHASE / EVENT	PAYMENT STATUS	ORDER STATUS	STOCK COUNT	CART STATUS
1. Order Created	pending_verification	payment_pending	Unchanged	Retained
2. Payment Authorized (Client Verify)	paid	preparing	Decrementd (-Qty)	Purged (Cleared)

PHASE / EVENT	PAYMENT STATUS	ORDER STATUS	STOCK COUNT	CART STATUS
3. Webhook (payment.captured)	paid	preparing	Guaranteed Decrement	Guaranteed Purge
4. Webhook (payment.failed)	failed	payment_pending	Unchanged	Retained for Retry
5. Order Shipped (Admin)	paid	shipped	Unchanged	N/A
6. Order Delivered	paid	delivered	Unchanged	N/A

09

Failure Modes & Mitigation Table

FAILURE SCENARIO	POTENTIAL RISK	CRAFTYWRAP ARCHITECTURAL SOLUTION
Client closes browser before callback	User is charged, but order remains stuck in payment_pending	Asynchronous Webhook: Razorpay delivers payment.captured server-to-server. Webhook independently confirms payment and triggers crafter preparation.
Network drops during webhook delivery	Razorpay receives timeout, order not confirmed	Retry Engine + Idempotency: Razorpay retries webhook with exponential backoff. WebhookEvent table prevents double processing on receipt.
Concurrent client verify & webhook hit	Stock decremented twice; promo count incremented twice	Condition Guard & Session Lock: isFirstPaymentVerification = (status !== 'paid') within an isolated MongoDB ACID session guarantees singular execution.
Tampered webhook payload	Attacker fakes paid event to get free items	Raw Body HMAC-SHA256: Computed using req.rawBody and timingSafeEqual against secret. Unauthenticated payloads return 400 Bad Request immediately.